yrr sare matrial and important information sab ko ek jgha add krke ek pdf bna kr dede taki agr ma dusre system pr hu ya net na aa rha ho to kam kr saku tb bhi or hai yrr thoda thoda and me presudo code bhi add kr dena hint ke liya you know begginer

Bhai, main directly `.pdf` file to generate karke nahi de sakta (AI limitations, you know!), lekin main ye pura material ekdum clean aur well-structured document ke format mein yahan de raha hoon.

Tum bas is screen par **Ctrl + P** (ya Mac par Cmd + P) dabana aur **"Save as PDF"** select kar lena. Tumhara offline document ekdum ready ho jayega! Sath hi maine end mein har project ke liye pseudo-code (hints) bhi add kar diye hain taki jab atko to thoda direction mil jaye.

# 🚀 JavaScript Mastery: Advanced Practice Projects

**Prerequisites:** DOM Manipulation, Event Delegation, Arrays/Objects, Promises, Fetch API, Async/Await.
**Target Timeframe:** 3 to 4 weeks (1-2 hours/day).

## 1. GitHub Profile Finder & Card Generator

**The Goal:** Build an app with a search bar where a user can type any GitHub username and hit enter to generate a beautiful profile card.
**Core Skills:** `async/await`, `fetch`, DOM element creation, Error handling (`try/catch`).

**UX/UI Blueprint:**

- **Search Area:** An input field and a magnifying glass button.
- **Profile Card (Component):** Displays Avatar Image, Username, Followers/Following/Repos count, and Bio.
- **Error State:** If the API returns a 404 (User Not Found), gracefully show a text message instead of a broken card.

**Beginner Hint & Pseudo-code:**

JavaScript

```javascript
// HINT: Use async/await to make the code cleaner than standard .then()
const searchBtn = document.getElementById('searchBtn');
const inputField = document.getElementById('usernameInput');
const cardContainer = document.getElementById('cardContainer');

searchBtn.addEventListener('click', async () => {
    const username = inputField.value;

    try {
        // 1. Fetch data from github API
        const response = await fetch(`https://api.github.com/users/${username}

        // 2. Check if user is not found (404)
        if (!response.ok) {
            throw new Error("User not found");
        }

        // 3. Convert to JSON
        const data = await response.json();

        // 4. Create DOM elements and inject data (Avatar, Name, etc.)
        cardContainer.innerHTML = `<img src="${data.avatar_url}"> <h2>${data.n

    } catch (error) {
        // 5. Handle the error (Show "User not found" on UI)
        cardContainer.innerHTML = `<p>${error.message}</p>`;
    }
});
```

## 2. Interactive "Sticky Notes" Task Manager

**The Goal:** A dashboard where you can add new sticky notes, edit the text inside them, and click a trash icon to delete them. Notes must survive a page refresh.
**Core Skills:** Arrays/Objects, Event Delegation, DOM Traversing ( `parentNode` ), LocalStorage.

**UX/UI Blueprint:**

- **Header:** A simple banner with an "Add Note" button.
- **Sticky Notes:** Square cards with an editable text area ( `<textarea>` ) and a small trash icon in the corner.
- **Storage Cue:** A small text indicating "Saved to LocalStorage".

**Beginner Hint & Pseudo-code:**

JavaScript

```
// HINT: Store your notes as an array of objects.
let notesArray = JSON.parse(localStorage.getItem('myNotes')) || [];

function saveNotes() {
    // Save array to local storage
    localStorage.setItem('myNotes', JSON.stringify(notesArray));
}

function renderNotes() {
    // Loop through notesArray and create HTML for each note
}

// Event Delegation for Deleting a note
const dashboard = document.getElementById('dashboard');

dashboard.addEventListener('click', (e) => {
    // Check if the clicked element is the trash icon
    if (e.target.classList.contains('trash-icon')) {
        // Find the ID of the parent note
        const noteId = e.target.parentNode.id;

        // Filter out this note from notesArray
        notesArray = notesArray.filter(note => note.id !== noteId);

        // Update LocalStorage and re-render the UI
        saveNotes();
        renderNotes();
    }
});
```

## 3. Dynamic Weather Dashboard

**The Goal:** A dashboard that fetches real-time weather data for a specific city using a free API (like OpenWeatherMap). The background changes based on the weather.
**Core Skills:** Complex JSON parsing, string interpolation, dynamic styling.

**UX/UI Blueprint:**

- **Search Input:** Input field for the city name.
- **Current Weather (Main Card):** Large display of Temperature, Location Name, Weather Description, Humidity, and Wind Speed.
- **Dynamic Background:** Use a `switch` statement based on weather conditions (Clear, Rain, Clouds) to change the `document.body.style.backgroundColor`.

**Beginner Hint & Pseudo-code:**

JavaScript

```
// HINT: Read the API JSON response carefully in console.log first!
async function getWeather(city) {
    try {
        const response = await fetch(`API_URL_HERE?q=${city}&appid=YOUR_KEY`);
        const data = await response.json();

        // Extract what you need
        const temp = data.main.temp;
        const condition = data.weather[0].main;

        // Update UI...

        // Change background based on condition
        switch(condition) {
            case 'Rain':
                document.body.style.backgroundColor = 'darkgray';
                break;
            case 'Clear':
                document.body.style.backgroundColor = 'skyblue';
                break;
            default:
                document.body.style.backgroundColor = 'white';
        }
    } catch(error) {
        console.log("City not found");
```

```
    }
}
```

## 4. Async Typing Speed Tester

**The Goal:** Display a random quote on the screen. The user has 60 seconds to type it out as accurately as possible. Calculate Words Per Minute (WPM).
**Core Skills:** `setInterval` / `clearInterval`, Arrays (comparing target text vs. user input), Math logic.

**UX/UI Blueprint:**

- **Top Bar:** Shows Time Left (60s countdown) and current WPM.
- **Target Text:** A paragraph where every character is wrapped in a `<span>`. As the user types, characters turn green (correct) or red (incorrect).
- **Input Box:** A `<textarea>` where the user types.

**Beginner Hint & Pseudo-code:**

JavaScript

```javascript
// HINT: You need a timer and a way to compare arrays index by index.
let timerInterval;
let timeLeft = 60;

function startTimer() {
    timerInterval = setInterval(() => {
        timeLeft--;
        document.getElementById('timerDisplay').innerText = timeLeft;

        if (timeLeft === 0) {
            clearInterval(timerInterval);
            calculateWPM();
            // Disable input field
        }
    }, 1000); // 1000ms = 1 second
}

// Compare characters
const inputField = document.getElementById('typingInput');
```

```
const quoteSpans = document.querySelectorAll('.quote-char'); // Assuming you c

inputField.addEventListener('input', () => {
    // Start timer on first keystroke
    if(timeLeft === 60) startTimer();

    const arrayValues = inputField.value.split(''); // Break input into array

    quoteSpans.forEach((characterSpan, index) => {
        const character = arrayValues[index];

        if (character == null) {
            // User hasn't typed this yet
            characterSpan.classList.remove('correct', 'incorrect');
        } else if (character === characterSpan.innerText) {
            // Typed correctly
            characterSpan.classList.add('correct');
            characterSpan.classList.remove('incorrect');
        } else {
            // Typed incorrectly
            characterSpan.classList.add('incorrect');
            characterSpan.classList.remove('correct');
        }
    });
```